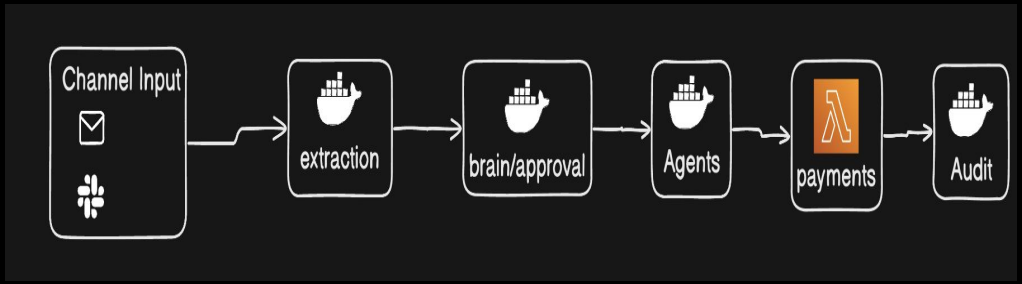
Problem & Vision

AI Accounts Payable Employee

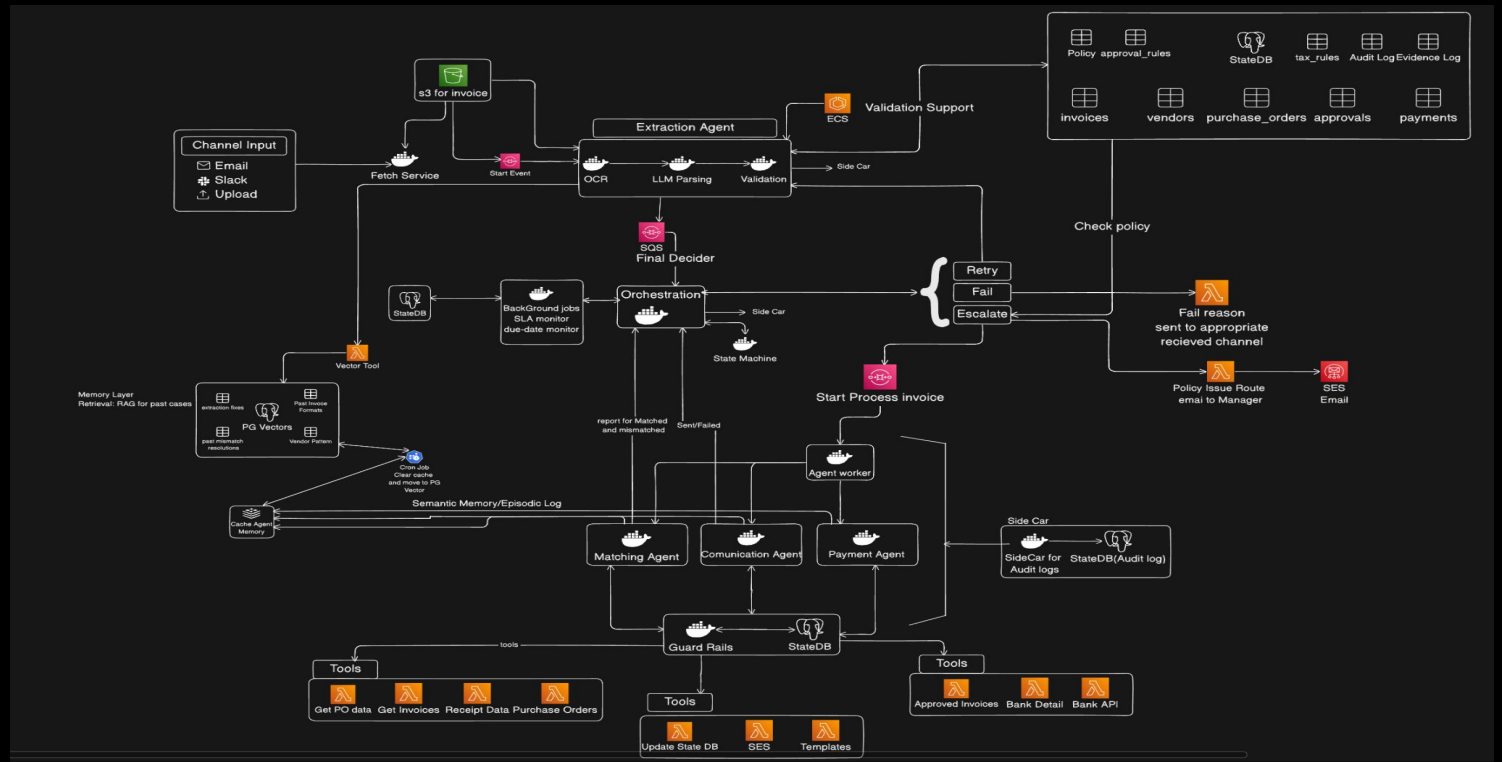
Manual AP is slow & error-prone

High compliance & fraud risk

Need end-to-end autonomous execution

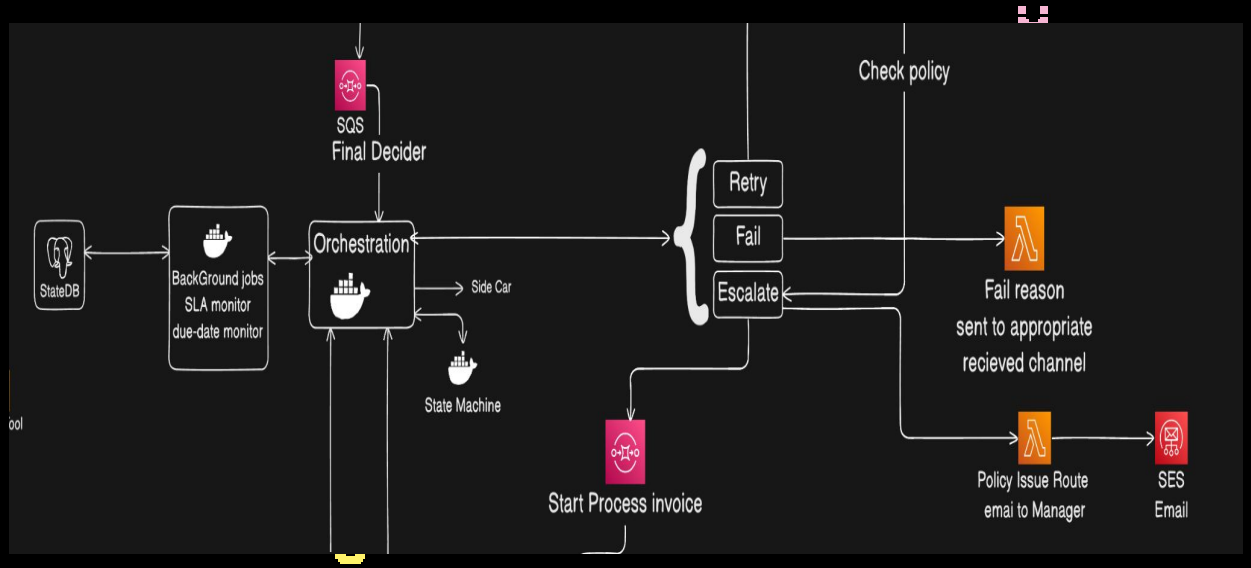


System Overview (High-Level Architecture)



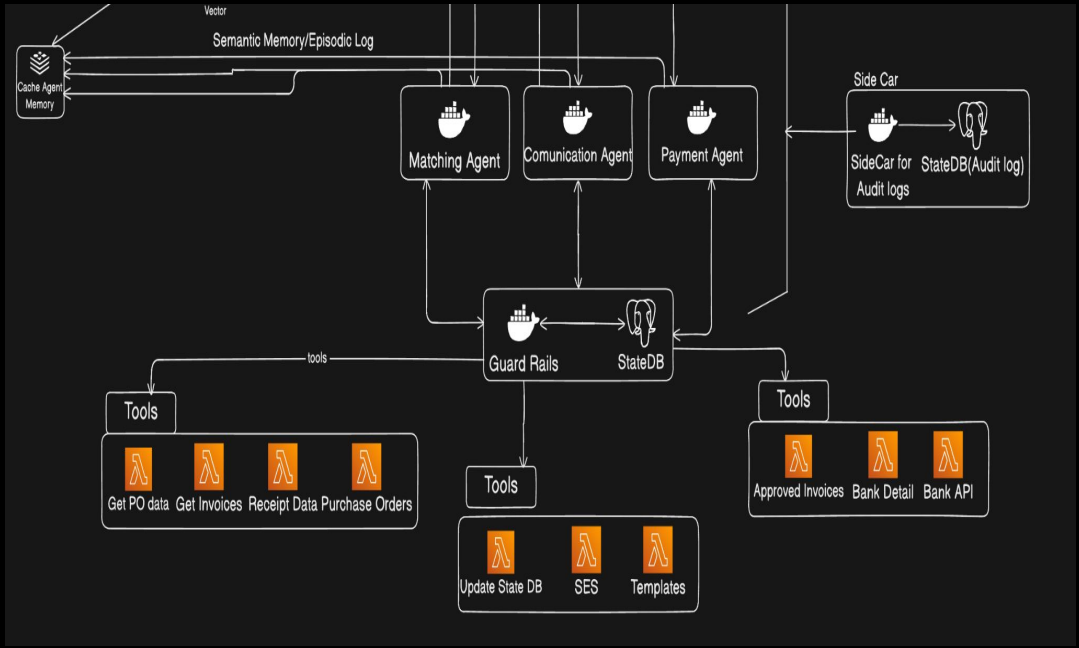
Workflow & State Machine

- Deterministic execution
- No hidden decisions

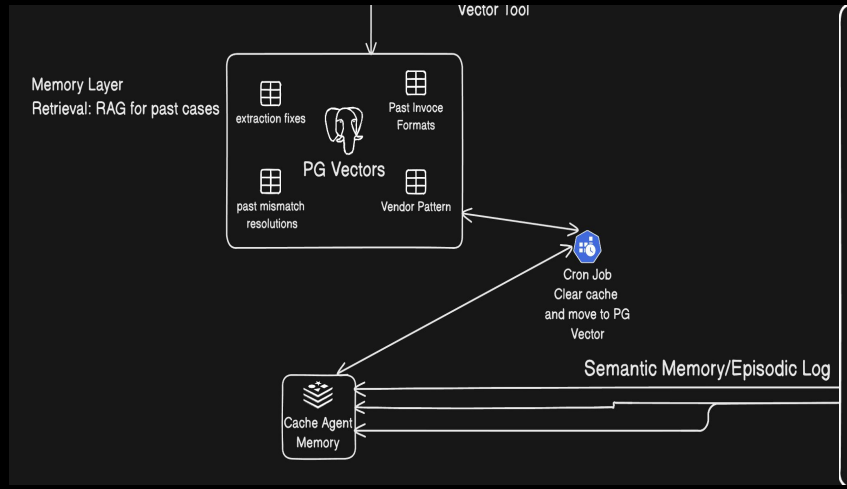


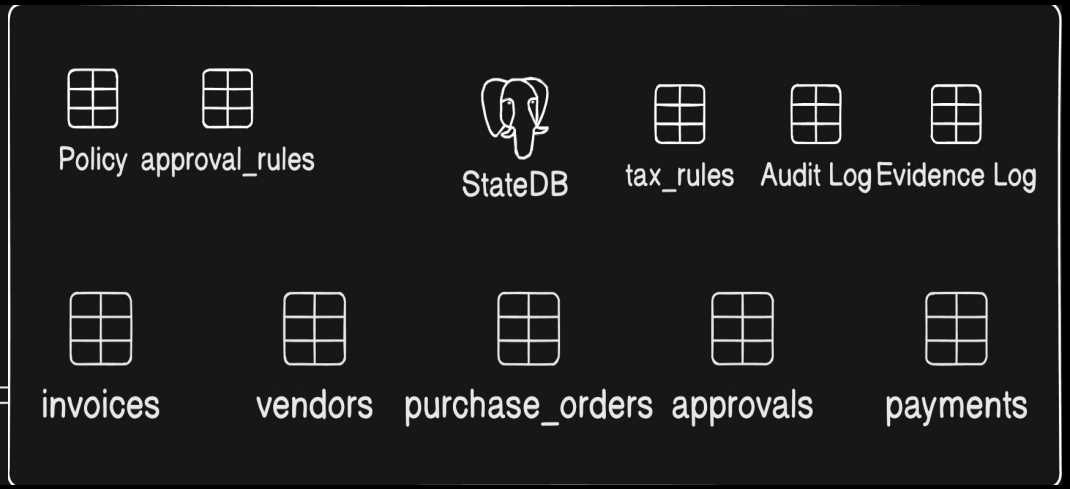
Multi-Agent Design

Agent	Responsibility
Communication	vendor/approval emails
Matching	PO/receipt reconciliation
Guardrails	safety checks
Payment	bank execution



Memory + Safety + Observability





- Structured truth
- Semantic learning
- Full audit trail

Technical Design Doc

1. Problem Framing

Background

Accounts Payable (AP) is the finance function responsible for receiving supplier invoices, validating them against purchase orders and receipts, ensuring policy and tax compliance, routing approvals, and executing payments. Although supported by modern accounting software, most AP workflows remain highly manual and operationally intensive.

Finance teams still spend significant time extracting data from PDFs and emails, reconciling mismatches, chasing approvals, handling exceptions, and performing payment checks. As invoice volumes grow, these manual steps become bottlenecks that slow down operations and increase the risk of errors and fraud.

Challenges in AP Automation

Automating AP is non-trivial due to the variability and risk involved in financial processes:

- Unstructured invoice formats (PDFs, scans, emails)
- Frequent edge cases (missing PO, mismatches, partial receipts)
- Strict compliance requirements (VAT, SOX, segregation of duties)
- Fraud risks (duplicate invoices, vendor bank detail changes)
- Organization-specific policies and approval chains
- Need for full auditability and explainability

Most existing “AI assistants” provide suggestions but still require constant human supervision and do not complete tasks end-to-end safely.

Goals

The goal is to design an **AI Accounts Payable Employee** capable of autonomously executing the AP workflow from intake to payment while maintaining strong safety and compliance controls.

The system should:

- Ingest invoices from multiple channels (email, uploads, APIs)
- Extract and validate invoice data automatically
- Perform PO/receipt matching and reconciliation
- Apply policy checks and route approvals
- Handle exceptions and vendor communication
- Schedule and execute payments safely
- Maintain complete audit logs and traceability

- Operate proactively with minimal human intervention
- Generalize across organizations using configurable policies and connectors

The system should behave like a reliable digital employee, not a chatbot.

Assumptions

- External systems (email, banking) expose APIs or connectors
- Business policies and approval rules are configurable
- A structured database serves as the source of truth
- AI is used for extraction, reasoning, and automation, while deterministic controls ensure safety
- Human escalation remains available for high-risk or ambiguous cases

2. Core Workflows

Happy Path (Standard Invoice Processing)

The system executes AP tasks through a deterministic orchestration loop.
Each invoice moves forward through explicit states until completion.

Flow:

```
Invoice received (email/upload)
→ Extraction (OCR + LLM parsing)
→ Validation (schema, VAT, duplicates)
→ PO/Receipt Matching
→ Policy Check
→ Approval (if required)
→ Payment Scheduling
→ Payment Execution
→ Completed
```

Execution Model:

```
Orchestrator → select agent → agent acts → guardrails verify → state updated → repeat
```

All steps are logged and auditable.

Exception Handling (Key Scenarios)

Failures do not break the workflow.
They transition the invoice into controlled states.

Scenario	System Action
Extraction failure	retry → escalate

Duplicate invoice	block payment → flag
Missing PO	notify vendor/manager
Matching mismatch	escalate for review
VAT/tax failure	reject + request correction
Approval delay (SLA breach)	auto-reminder → escalate
Vendor bank change	fraud check + mandatory approval
Tool/API failure	retry with backoff

State Machine

Invoices move through predefined states:

```
RECEIVED → EXTRACTED → VALIDATED → MATCHED
→ APPROVAL_PENDING → APPROVED → PAID → COMPLETED

Exceptions:
RETRY | ESCALATED | FAILED
```

The Orchestrator alone controls transitions to ensure deterministic and safe execution.

3. Agent Loop + State Machine / Orchestration Approach

Orchestration Model

The system follows a **supervisor–worker pattern**.

The **Orchestrator (brain)** controls all decisions, while **agents (workers)** execute tasks.
Agents never determine workflow or policy outcomes.

This ensures deterministic behavior, safety, and easy debugging.

Agent Loop

Each invoice is processed through a repeated control loop:

```
read current state
→ evaluate policy/rules
→ select next agent
→ execute task
→ guardrail checks
→ update state
→ log evidence
→ repeat until complete
```

If a step fails, the Orchestrator retries, escalates, or routes to human review.

Agent Execution Pattern

- Each agent follows: Receive task → Plan action → Execute tools → Validate result → Update state OR retry/escalate
- Success criteria defined per task (e.g., extraction = all required fields present)
- Max 3 retries, then escalate with reason

State Machine

Workflow progression is modeled as an explicit state machine to prevent ambiguous or unsafe transitions.

```
RECEIVED
→ EXTRACTED
→ VALIDATED
→ MATCHED
→ POLICY_CHECK
→ APPROVAL_PENDING
→ APPROVED
→ PAYMENT_SCHEDULED
→ PAID
→ COMPLETED

Exceptions:
RETRY | ESCALATED | FAILED
```

Only the Orchestrator can modify state.

Responsibilities Split

Component	Responsibility
Orchestrator	decisions, routing, recovery
Agents	task execution only
Guardrails	safety & compliance checks
StateDB	source of truth

This design guarantees safe, autonomous, and observable execution across all invoices.

4. Tooling Layer (Connectors) & Permission Model

Tooling / Connectors Layer

All external integrations are abstracted through a dedicated **Connectors layer**.
Agents never directly access third-party systems. Instead, they invoke standardized tools that encapsulate external APIs.

This design ensures portability, safety, and easy integration across organizations.

Supported connector types:

- Email (Gmail/SES)
- Banking/Payment APIs
- Document storage (S3/Drive)
- Messaging (Slack)

Execution flow:

`Agent → Guardrails → Connector → External System`

This prevents unsafe or unauthorized operations.

Permission Model

Financial actions require strict separation of duties and controlled access.

The system enforces permissions through:

- Role-based access control
- Policy checks
- Approval gates

All tool calls are validated against permissions and policies before execution.

Design Principles

- Tools are stateless and replaceable
- Connectors are configurable per organization
- No hardcoded integrations
- Policies determine behavior, not code

This enables the system to generalize across different companies with minimal setup.

Company Onboarding Process:

- Upload config file with approval thresholds, policy rules
- Connect integrations via OAuth (Gmail/Slack)
- Map invoice fields to system schema
- Test with 5 sample invoices → go live (takes <1 hour)

Policy Engine:

- Policies stored as rules in `policy_approval_rules` table
- Evaluated before each state transition
- Humans can override with justification (logged in audit trail)

Why Generalizable:

- No hardcoded workflows
- Policies = data, not code
- Tool connectors are swappable
- Multi-tenant by design

5. Memory Strategy & Context Management

The system maintains multiple forms of memory to balance reliability, learning, and cost efficiency.
Financial correctness relies on structured storage, while AI reasoning leverages semantic retrieval and historical context.

Memory is intentionally separated into three layers.

Structured Memory (Source of Truth)

Implemented using Postgres (StateDB).

Stores all deterministic financial data:

- invoices and line items
- vendors and bank details
- purchase orders and receipts
- approvals and workflow states
- policies and configurations
- audit logs

This database is the single source of truth.
All agents read and write only through this layer.

Semantic Memory (Contextual Retrieval)

Implemented using a vector database.

Stores:

- past invoice formats
- vendor-specific patterns
- historical resolutions
- extraction corrections
- similar past cases

Used by:

- Extraction Agent

- Matching Agent

This improves accuracy over time without affecting financial correctness.

Episodic Memory (Execution History)

Stored as structured audit logs.

Records:

- action taken
- actor (agent/human/system)
- inputs/outputs
- decision reason

Used for:

- traceability
- learning and reflection

Context Principles

- Structured memory = facts
- Semantic memory = experience
- Episodic memory = history
- No long LLM context retention
- Retrieve only task-relevant information

This avoids context blowup while enabling continuous improvement across runs.

Reflection Mechanism:

- System analyzes failed/escalated cases weekly
- Identifies error patterns (e.g., "vendor X invoices always missing PO")
- Updates extraction prompts or semantic memory
- Example: If 5 invoices from same vendor fail, flag vendor profile for review

6. Observability, Auditability & Cost Controls

Financial automation requires full transparency and traceability.
Every action taken by the system must be explainable, reversible, and compliant.

The platform is designed with observability and safety as first-class concerns rather than afterthoughts.

Observability

All components emit structured logs and execution traces.

Captured for every step:

- current state
- agent invoked
- tool used
- input/output summary
- latency
- errors/retries

This enables:

- real-time monitoring
- performance tracking
- operational reliability

Workflows can be replayed and inspected deterministically.

Auditability

All actions are recorded in an immutable **audit log** stored in the StateDB.

Each record includes:

- who (agent/human/system)
- what action
- why (reason/policy/decision)
- when (timestamp)
- before/after values

This provides:

- financial compliance (SOX/GDPR/VAT)
- approval traceability
- fraud investigation support
- payment justification

Every payment or decision can be fully explained.

SLA Monitor (runs every 15 min):

- Checks invoices with due_date <48hrs AND status ≠ approved
- Sends Slack/Email reminder to approvers
- Auto-prioritizes in approval queue

Guarded Execution

Sensitive operations pass through guardrails before execution:

```
Agent → Guardrails → Connector
```

Checks include:

- approval present
- duplicate detection
- vendor risk validation
- amount thresholds
- permission checks

Unsafe actions are blocked or escalated automatically.

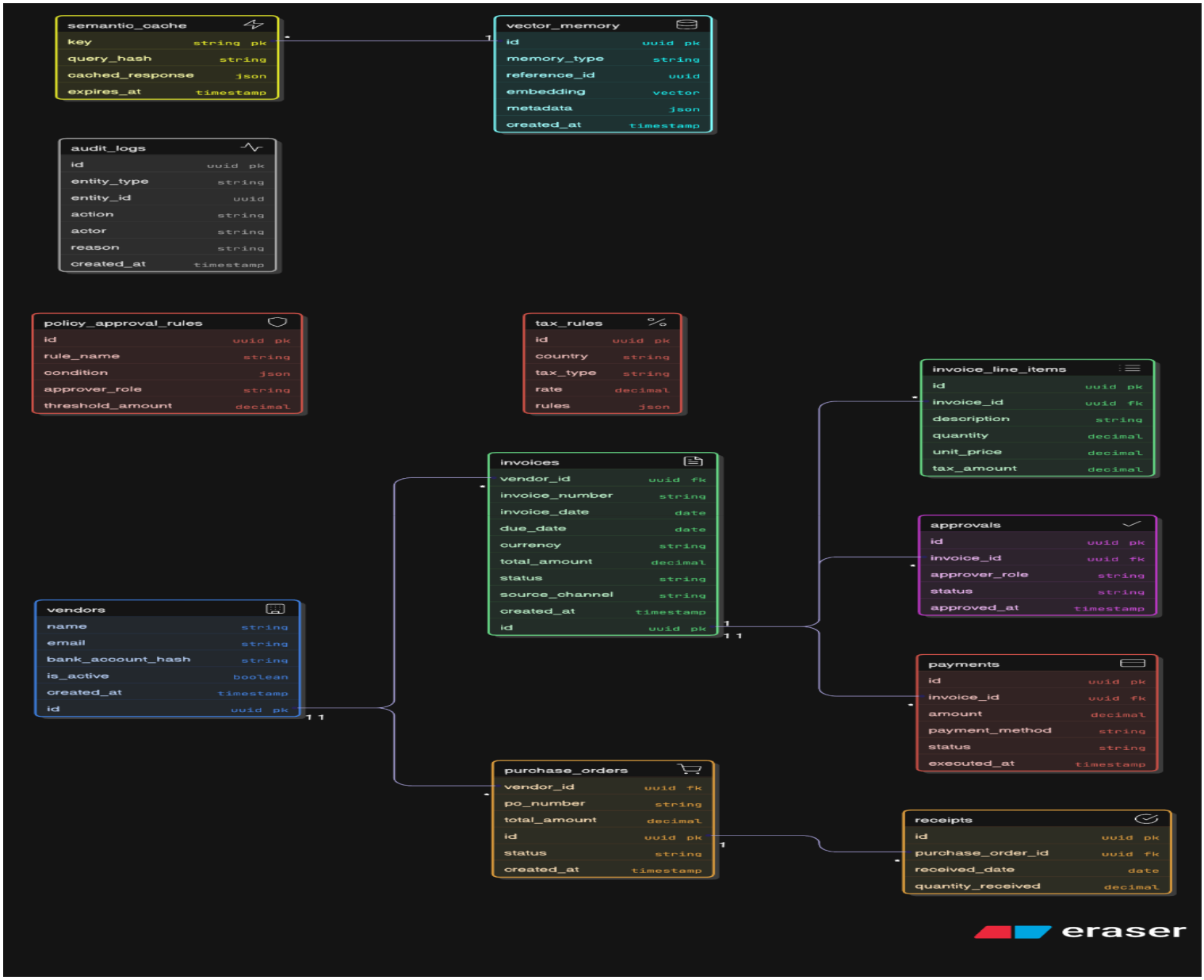
Cost Controls

To ensure scalable operation:

- retry limits
- caching of repeated lookups
- selective semantic retrieval (no large contexts)
- background batching for non-urgent tasks

This keeps inference and infrastructure costs predictable.

7. ERD Diagram



ERD – Summary

vendors (PK: id) – supplier master; parent of invoices and purchase_orders
invoices (PK: id, FK: vendor_id) – core invoice; parent of line_items, approvals, payments
invoice_line_items (PK: id, FK: invoice_id) – invoice charge breakdown
purchase_orders (PK: id, FK: vendor_id) – approved spend records
receipts (PK: id, FK: purchase_order_id) – delivery confirmation for 3-way match
approvals (PK: id, FK: invoice_id) – approval decisions per invoice
payments (PK: id, FK: invoice_id) – payment execution records

policy_approval_rules (PK: id) – configurable approval logic
tax_rules (PK: id) – VAT/tax validation rules
audit_logs (PK: id) – action history for traceability

vector_memory (PK: id) – semantic embeddings linked to invoices/cases
semantic_cache (PK: key) – Redis cache for fast retrieval (ephemeral)

Relationships

Vendor → many Invoices, Purchase Orders
Invoice → many Line Items, Approvals, Payments
Purchase Order → many Receipts
Semantic Cache → references Vector Memory

ERD – Design Rationale

This schema models the Accounts Payable workflow around **invoices as the central entity**, with related tables capturing vendors, matching (PO/receipts), approvals, and payments to reflect the real financial lifecycle.

Structured tables ensure **deterministic state, auditability, and compliance**, while vector memory and Redis cache provide **learning and performance optimization without affecting financial correctness**.

8. MVP Build Plan (4 Weeks)

Week 1 — Core Infrastructure & Intake

- Setup Postgres (StateDB) + schema
- Build Orchestrator loop + state machine
- Implement Intake + Extraction agents (email → OCR/LLM → invoice)
- Store invoices and logs
- Basic happy path working locally

Output: invoice ingestion → structured data stored

Week 2 — Validation & Matching

- Validation agent (schema, VAT, duplicate checks)
- PO/Receipt matching agent
- Policy tables + simple policy checks
- Exception states + retries/escalations

Output: invoice validated and matched automatically

Week 3 — Approvals & Payments

- Approval routing + SLA monitor
- Communication agent (email/Slack)
- Payment agent + mock/bank connector
- Guardrails before payments

- Audit logs for all actions

Output: approval → payment end-to-end flow

Week 4 — Safety, Memory & Polish

- Vector memory + Redis semantic cache
- Background jobs (SLA/due-date monitors)
- Observability + structured logs
- Tests for core scenarios
- Demo UI + sample scenarios

Observability dashboard: Invoice states chart, error rate by agent, avg processing time, trace viewer (click invoice → see all actions)